

Designing a Novel Optimizer for Multi-Task CNNs on CIFAR-10

Introduction

Multi-task learning with CNNs (e.g. simultaneously performing classification, segmentation, and detection) poses unique optimization challenges. Standard optimizers like **Nadam** (Adam with Nesterov momentum) provide strong performance on single tasks, but a new optimizer can potentially outperform Nadam across multiple tasks by addressing multi-task specific issues. We aim to design an optimizer from scratch that improves **training speed, accuracy, generalizability, and memory efficiency** for a CNN on CIFAR-10 multi-task objectives. This report explores core principles of optimizer design – from adaptive learning rates and momentum variants to second-order methods and meta-learning – and proposes a novel optimization algorithm leveraging these principles. We also outline an implementation architecture and evaluation strategy, citing state-of-the-art research for guidance.

Challenges in Multi-Task Learning Optimization

Training a single CNN to perform diverse tasks (classification, segmentation, detection) means optimizing a **combined loss** with gradients from each task. Key challenges include:

- **Gradient Interference:** Gradients from different tasks can conflict or compete, slowing convergence or favoring one task at the expense of others. Effective multi-task optimizers must mitigate such conflicts ¹ ².
- **Differing Scales:** Loss magnitudes and gradient norms can vary by task. Without adjustment, a task with larger gradients can dominate the updates, leading to suboptimal accuracy on other tasks ³. Balancing task contributions is essential for accuracy and generalization across all tasks.
- **Efficiency Trade-offs:** Multi-task training can be slower per epoch (due to computing multiple losses) and more memory-intensive (multiple output heads, larger batch needed to cover tasks). The optimizer should accelerate convergence (fewer epochs) and minimize overhead memory usage.
- **Generalization:** An optimizer must find parameter updates that not only minimize training loss but also generalize well on all tasks. This is challenging when tasks have different notions of what constitutes a "good" solution. For example, a very low classification loss might come at the cost of a slightly higher segmentation loss – the optimizer should navigate such trade-offs to find a Pareto-optimal balance.

These challenges mean that simply applying a single-task optimizer to a summed multi-task loss may be suboptimal. We need to build in mechanisms for **adaptive learning, gradient normalization, and conflict avoidance** to handle the multi-task scenario.

Goals for a Novel Optimizer

To outperform Nadam on multi-task CIFAR-10, our optimizer should target improvements on several fronts:

- **Faster Convergence:** Achieve lower loss or higher accuracy in fewer training iterations than Nadam. This can be measured by epochs or time to reach a certain accuracy on CIFAR-10. Faster convergence speeds up experimentation and reduces compute cost ⁴.
- **Higher Accuracy Across Tasks:** Deliver better final performance on each task (e.g. higher classification accuracy, higher segmentation IoU, better detection mAP) than Nadam. Importantly, none of the tasks should significantly lag behind the others – the optimizer should improve the *average* performance and ideally each individual task's performance.
- **Better Generalization:** Produce models that generalize to unseen data, avoiding overfitting on any task. An optimizer can promote generalization by finding flatter minima or by regularizing updates (e.g. via weight decay or sharpness-aware updates). We will seek an algorithm that, like SGD, doesn't compromise generalization for speed ⁵.
- **Memory Efficiency:** Use less memory for optimizer states than Nadam/Adam (which store momentum and second moment of comparable size to model parameters). Lower memory footprint allows larger batch sizes or models on the same hardware. The target hardware is a single RTX 3060 (12GB); the optimizer must fit comfortably in this budget. Techniques like state compression or fewer state tensors will be considered to minimize memory overhead ⁶ ⁷.

With these goals defined, we explore foundational ideas to guide the optimizer design.

Core Algorithmic Innovations and Principles

To build a novel optimizer, we consider several key principles and recent innovations:

Adaptive Learning Rates

Adaptive gradient methods adjust the step size per parameter based on the history of gradients. This helps fast convergence by taking larger steps in dimensions with small gradients and smaller steps where gradients are large. Notable techniques include:

- **Adagrad/RMSprop/Adam:** Adagrad accumulates squared gradients to shrink learning rates on frequently updated parameters. Adam (and RMSprop) maintain an exponential moving average of squared gradients to adapt step sizes, combined with momentum on the first moment ⁵. These methods excel at quick initial progress and handling different feature scales, which can benefit multi-task learning where some tasks generate larger gradients than others. However, vanilla Adam can generalize worse than SGD if not properly regularized ⁸.
- **AdamW and Decoupled Weight Decay:** Decoupling weight decay from the gradient update (AdamW) improved generalization by treating weight decay as a separate step, preventing the adaptive learning rate from inadvertently canceling out regularization ⁹. Our optimizer will similarly include weight decay in a principled way to aid generalization.
- **AdaBelief:** An Adam variant that adapts step sizes by how much the gradient *deviates* from an expected value. If the observed gradient differs greatly from the running average (i.e., is surprising), AdaBelief considers it unreliable and reduces the step; if it aligns with expectations, it increases the step ¹⁰. This approach yielded **fast convergence like Adam while achieving accuracy**

comparable to SGD on ImageNet and improved stability on tasks like GAN training ¹¹. Such adaptive trust in gradients could help stabilize multi-task training, where gradient noise might differ per task.

- **Learning Rate Schedules:** While not part of the optimizer algorithm per se, dynamic LR schedules (cyclical learning rates, cosine decay, etc.) can significantly impact training speed and generalization. Our design will be compatible with scheduling (e.g. warm-up phases for detection tasks that often require it), but the core focus is on the optimizer's internal adjustments rather than predefined schedules.

Design Implication: We will incorporate per-parameter adaptive step sizing to ensure efficient learning on all tasks. A promising choice is to use an Adam-like update rule (for its fast progress) but modify its second-moment calculation (e.g. using AdaBelief's "belief" idea or clipping untrustworthy updates). This retains the benefit of adaptation while avoiding some of Adam's pitfalls in generalization.

Momentum and Acceleration

Momentum methods accumulate a velocity for parameters to smooth out noisy gradients and accelerate through shallow or narrow valleys. Our optimizer can leverage momentum to improve both speed and stability:

- **Classic Momentum and Nesterov:** Standard momentum (a fraction β of the previous update plus current gradient) helps dampen oscillations. Nesterov momentum looks a step ahead by computing gradient at the projected next position, often improving convergence. Nadam already adds Nesterov correction to Adam. We will retain Nesterov-style momentum for its proven benefit in speed.
- **Lookahead:** A recent innovation where two sets of weights ("fast" and "slow") are maintained ¹². Fast weights are updated k times with a base optimizer; then the slow weights are moved slightly toward the fast weights, and the fast weights are reset to slow. Lookahead has shown to **smooth the optimization trajectory and improve final generalization**, even enabling training without batch normalization in some cases ¹³ ¹⁴. We can integrate lookahead on top of our core update rule to stabilize training across tasks that may have very different convergence speeds.
- **Aggregated Momentum (AggMo):** Instead of a single momentum buffer, AggMo uses multiple momenta with different decay rates and averages them ¹⁵. This can capture short-term and long-term trends in gradients simultaneously. While AggMo is more of a research curiosity, the idea of multi-timescale momentum could help in multi-task scenarios (short-term momentum might respond quickly to one task's sudden gradient change, while long-term momentum carries overall direction).
- **Sign Momentum (Lion Optimizer):** A radically different momentum-based approach is used by Lion ("Evolved Sign Momentum"). Lion maintains a momentum vector but updates parameters using the **sign of the momentum** rather than the raw value ¹⁶. This means each parameter update has a uniform magnitude (scaled by a learning rate) and only the direction (sign) is taken from the momentum-cum-gradient estimate. By doing so, Lion forgoes precise gradient magnitude information, which reduces noise and significantly **reduces memory** (no second-moment needed) ⁶. Despite its simplicity, Lion was discovered via meta-learning to perform well on vision and language tasks ¹⁷ and even improves ImageNet accuracy over AdamW in some cases ¹⁸. This concept suggests that in some regimes, just the sign of an update is enough to make progress, and it inherently limits any single step's magnitude – which might benefit generalization (preventing overly large parameter jumps). We will consider a **sign-based update component** for memory

efficiency and robustness, possibly hybridized with magnitude-based updates (e.g., using sign for direction but still scaling by an adaptive factor).

Design Implication: Our optimizer will include a momentum term (with Nesterov correction). We may experiment with *sign-based updates* à la Lion to limit step sizes uniformly, while using adaptive scaling to reintroduce magnitude information carefully. Additionally, we plan to use a **lookahead** mechanism as an outer loop to ensure the combined multi-task loss is moving in a smooth, improving direction. Momentum hyperparameters (β values, number of lookahead steps, etc.) will be tuned for a balance of speed and stability.

Second-Order Information (Curvature)

First-order methods like SGD or Adam use gradient information but ignore the curvature (second derivative/Hessian) of the loss surface. Incorporating curvature can significantly accelerate convergence by taking more informed steps (closer to Newton's method), but full second-order methods are computationally expensive and memory-heavy. Recent research offers **approximate second-order optimizers** that could inspire our design:

- **Quasi-Newton & L-BFGS:** Algorithms like L-BFGS approximate the Hessian inverse through past updates, achieving superlinear convergence in classical settings. However, in deep learning, they are less common due to noisy gradients and extremely high-dimensional parameter spaces. For our single-GPU scenario, full L-BFGS is impractical, but ideas like periodically using a few past gradients to adjust step sizes might be borrowed.
- **K-FAC (Kronecker-Factored Approximate Curvature):** K-FAC approximates the Fisher information matrix (a form of Hessian) for each layer in a factorized way, and uses its inverse to precondition gradients. It has seen success speeding up training in CNNs and deep reinforcement learning. The memory cost is significant (storing factors for each layer), and updating these factors is complex, especially for tasks like detection with many parameter layers. We note K-FAC as a background idea but seek lighter alternatives.
- **Shampoo:** An optimizer that computes a second-order preconditioning by forming large matrices of gradient correlations for each layer (splitting dimensions) and periodically updating their inverse (often via matrix square root) to precondition gradients. It yields faster convergence in some cases but at a high memory/computation cost (not ideal for 12GB GPU unless the network is very small). Newer **Scalable Shampoo** variations attempt to limit this overhead.
- **Sophia (Second-order Clipped Stochastic Optimizer):** Sophia is a 2023 optimizer designed for large models that uses an **inexpensive stochastic estimate of the Hessian diagonal** as a preconditioner ¹⁹. Every k steps (e.g. $k=10$), Sophia estimates the diagonal of the Hessian by either Hutchinson's method (random vector probe) or a Gauss-Newton approximation, which only adds ~5% compute overhead ²⁰. It then adjusts the update: parameters are moved in the direction of a **momentum-adjusted gradient divided by this Hessian diagonal estimate**, and the update is *clipped* to avoid oversized steps ²¹ ²². This approach essentially scales down updates in directions with high curvature (steep loss) and allows larger steps in flat directions. The clipping guards against any error in the curvature estimate or rapid curvature changes. Notably, Sophia achieved **50% fewer steps to reach the same loss as Adam** in language model pre-training, with no increase in memory and only modest per-step overhead ⁴. This demonstrates the potential of second-order information to dramatically speed up training. For our scenario, a lightweight diagonal Hessian approximation could help navigate the loss landscape of multiple tasks more efficiently than first-order methods.

- **AdaHessian:** Another diagonal Hessian-based optimizer that directly calculates a diagonal Hessian approximation via backpropagation of gradients (using second-order gradients). It adapts the learning rate for each parameter based on curvature and has shown improved convergence on some vision tasks. It requires computing second derivatives, which can be expensive, but for smaller CNNs on CIFAR-10 this might be feasible occasionally.
- **Sharpness-Aware Minimization (SAM):** While not a second-order optimizer per se, SAM is a technique that seeks flatter minima by performing an inner maximization of loss (within a small neighborhood of weights) for each update. The optimizer then takes a step that minimizes loss *and* ensures it's in a flat region. SAM can be combined with any optimizer (including our proposed one) to improve generalization by avoiding sharp minima. We mention it as a possible add-on during evaluation if generalization is a concern.

Design Implication: We intend to incorporate **curvature awareness** without heavy overhead. A practical way is to maintain a **running estimate of the Hessian diagonal or an adaptive preconditioner**. For example, we could extend the second moment vector (v in Adam) to approximate the curvature: instead of simply tracking $\mathbb{E}[g^2]$, track a blend of gradient variance and curvature. Using Sophia's idea, every few iterations we might do a cheap Hessian diagonal update and adjust our per-parameter learning rates. This should allow our optimizer to take aggressive steps where the loss landscape is flat (speeding up convergence) and cautious steps where the landscape is steep (preventing divergence), thereby improving training speed and potentially final accuracy ²¹. Any such second-order module must be constrained to low memory (just a vector of size equal to parameters and maybe a few extra buffers) and modest compute (5-10% overhead, which is acceptable on a single GPU).

Meta-Learning and Learned Optimizers

Instead of manually designing every aspect of the optimizer, an emerging idea is to **learn the optimizer itself** using meta-learning. In a meta-learning setup, one trains a small neural network or uses algorithm search to discover update rules that perform well on a class of tasks. Some relevant insights:

- **Learned Optimizer (L2O):** Andrychowicz et al. (2016) demonstrated training an LSTM to output update steps, effectively learning an optimizer that can train small neural nets faster than hand-designed methods on certain problems. While powerful, these learned optimizers are often specialized to the meta-training distribution and can be unstable or slow to train. They also might be impractical to run for large models (the learned optimizer adds overhead per parameter).
- **HyperNetworks for Optimizer:** Wichrowska et al. (2017) and others improved L2O by incorporating features like multi-scale updates and conditioning on training progress. These approaches show that optimizers can be tuned to outpace Adam on some tasks, but designing a generally superior learned optimizer remains a research challenge.
- **Symbolic Optimization Discovery:** Recent work by Chen et al. (2023) took a different approach – using symbolic program search to find optimization formulas. This is how the **Lion optimizer** was discovered ²³. The search algorithm evolved update rules and found that using a momentum and sign-based update was particularly effective. The success of Lion demonstrates that automated discovery can yield novel, simpler optimizers that might not be intuitive to humans. It also hints that there may be other undiscovered update rules that could excel in multi-task settings.
- **Hypergradient Descent:** Another meta-approach is to let the optimizer's hyperparameters (like learning rate, momentum coefficient) be *learnable*. Hypergradient descent computes the derivative of the loss with respect to a hyperparameter (e.g. learning rate) and updates the hyperparameter online ²⁴. For instance, the learning rate can be adjusted at each step based on whether the loss is

decreasing or oscillating. This technique can be wrapped around a base optimizer to make it self-tune. It's simple and has been shown to make training less sensitive to initial hyperparameter choices ²⁵. In a multi-task scenario, one could even imagine separate hypergradients for each task's learning rate or loss weight, learned as training progresses.

Design Implication: Fully training a learned optimizer for our problem (multi-task CIFAR-10) might be overkill, but we can still be inspired by meta-learning. We will incorporate **hyperparameter self-adjustment** – for example, gradually adjusting learning rate or task-wise loss weights based on observed gradients (a form of hypergradient descent). We will also ensure our design is **modular**, making it possible to plug in new update rules if future research (or automated discovery) suggests a better formula. In summary, the optimizer will have a few tunable components (momentum factor, etc.) that could even be adjusted on the fly. The overall architecture is informed by human insight and existing research, but leaves room for learning or tuning these components during training.

Gradient Balancing for Multi-Task Objectives

Beyond the core optimizer mechanics, multi-task learning benefits from explicit handling of multiple losses and gradients:

- **Gradient Normalization (GradNorm):** This technique dynamically rescales each task's loss gradient to balance training speed among tasks ³ ²⁶. If one task's loss is decreasing too slowly (or its gradient norm is too small), GradNorm increases that task's gradient weight, and vice versa. Chen et al. (2018) showed GradNorm can **improve accuracy and reduce overfitting across tasks** by preventing any task from dominating the updates ³. It essentially adjusts the learning rate per task to keep all tasks on a similar training pace.
- **Gradient Surgery (PCGrad):** PCGrad (Yu et al., 2020) explicitly addresses conflicting gradients by projecting task gradients onto the normal plane of each other when they conflict ¹ ²⁷. In practice, one can modify the gradients before the optimizer update: for each pair of tasks, if their gradients have a negative dot product (conflict), remove the component of one gradient that opposes the other. This “surgery” ensures that the update does not significantly harm any task. **PCGrad yielded substantial improvements in multi-task learning**, e.g. in multi-task CIFAR and multi-objective scene understanding, it improved data efficiency, optimization speed, and final performance ². It's a simple plug-in method that can work with any optimizer.
- **Conflict-Averse Gradient Descent (CAGrad):** A more recent extension that formulates multi-task optimization as finding a descent direction that is as good as possible for all tasks, using a negotiable compromise (minimizing worst-task improvement). CAGrad automatically balances objectives and has convergence guarantees ²⁸ ²⁹. This is more complex to implement, but it's good to be aware of as a theoretical direction.
- **Dynamic Task Weighting:** Methods like uncertainty weighting or loss weighting by inverse training rate can also balance tasks. These aren't gradient modifications per se, but adjusting loss weights for each task can indirectly shape the gradients that the optimizer sees. We mention this because an optimizer could incorporate a rule to adjust task weights over time (for example, reducing the weight of an already mastered task to focus on others).

Design Implication: To make our optimizer truly multi-task-aware, we will integrate a **gradient balancing step**. A practical choice is to use **PCGrad** due to its simplicity and demonstrated efficacy. Before each parameter update, we will adjust the task-specific gradients to remove conflicts, then sum them into one resultant gradient for the optimizer step. This can be combined with adaptive methods (e.g., do PCGrad,

then feed the combined gradient into our momentum/Adam update). We will also experiment with a form of GradNorm: monitoring the magnitudes of task gradients and potentially scaling the loss weights or gradients to prevent any task from lagging far behind. These mechanisms ensure that the optimizer's improvements in speed and accuracy apply **across all tasks**, not just the easiest one. By handling gradient conflicts, we expect more stable and faster optimization on the multi-task loss ³⁰.

Proposed Optimizer Design

Based on the above principles, we propose a new optimizer (let's call it **MTOpt** for Multi-Task Optimizer) with the following key features and steps each iteration:

1. **Compute Per-Task Gradients:** Compute gradients for each task loss $L_1, L_2, \dots, L_T$ with respect to model parameters θ . For a CNN on CIFAR-10, this might involve gradients from classification loss, segmentation mask loss, and detection loss.
2. **Gradient Balancing (PCGrad & Norm):** Adjust these gradients before combining:
3. For each pair of tasks (i, j) , if $\nabla L_i \cdot \nabla L_j < 0$ (conflict), project ∇L_i to $\nabla L_i - \frac{\nabla L_i \cdot \nabla L_j}{|\nabla L_j|^2} \nabla L_j$ ³¹ ²⁷. This removes the component of i 's gradient that opposes j 's gradient. Iterate or average across tasks to get a set of modified gradients $\{\tilde{g}_i\}$.
4. Optionally, rescale $\tilde{g}_i$ for each task using GradNorm: e.g., $\tilde{g}_i := \tilde{g}_i \cdot w_i$, where w_i is chosen such that $w_i \cdot |\tilde{g}_i|$ meets a balance criterion (all tasks' gradient norms are brought closer together or follow a proportional schedule) ³² ³³.
5. Finally, combine gradients (assuming tasks are summed in the loss): $g := \sum_{i=1}^T \tilde{g}_i$. This g is the final gradient of the multi-task loss after mitigating conflicts and imbalance.
6. **Momentum Update:** Maintain a momentum vector m of the same size as θ . Update it as:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$
 where g_t is the balanced gradient from step 2 at iteration t , and β_1 is the momentum factor (e.g. 0.9). This is similar to Adam's first moment update. We will use **Nesterov correction**, meaning when we use m_t for the parameter update, we actually compute the gradient at the lookahead position $\theta_{t-1} + \beta_1 m_{t-1}$ (this detail can be handled implicitly by adjusting how g_t is calculated if needed, as in Nadam). The momentum smooths the update direction over iterations.
7. **Adaptive Learning Rate (Second Moment or Curvature):** Maintain a second moment or curvature estimate vector v for adaptive scaling. We have options here:
8. **AdaBelief-style:** Update second moment v_t by the *squared deviation* of gradient: $v_t = \beta_2 v_{t-1} + (1 - \beta_2)(g_t - m_t)^2$. (Where $(g_t - m_t)^2$ is elementwise square of the difference between current grad and momentum, representing unexpected gradient component.)
9. **Adam-style:** $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ (elementwise square).
10. **Sophia-style (curvature):** Every k steps, instead of using g_t^2 , compute a Hessian diagonal estimate $h_t \approx \nabla^2 L(\theta)$ (diagonal). For example, use a Hutchinson estimator: sample a random vector r with Rademacher entries, compute Hr via autodiff (which is one

backward pass given $\mathbf{r}^{\text{top}} \nabla g$), and take the square of that as an unbiased diag Hessian estimate ³⁴. Or compute a Gauss-Newton diag by random label perturbation ³⁵. Then update $\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2) \mathbf{h}_t$. During steps between these full updates, $\mathbf{v}_t$ could remain the same or be slightly updated by gradient info.

11. **Adaptive Schedule:** Whichever method is used, we will incorporate an ϵ for numerical stability and possibly a cap/clipping on $\mathbf{v}_t$ to avoid extreme values. The outcome is a vector $\mathbf{v}_t$ that encodes per-parameter magnitude or curvature, used to adjust step sizes.
12. **Compute Parameter Step (Update Direction and Size):** We combine information from $\mathbf{m}_t$ and $\mathbf{v}_t$ to determine the update for parameters $\Delta \theta_t$. Two possible strategies could be toggled or hybridized:
13. **Adam/Nadam style:** $\Delta \theta_t = -\eta \frac{\mathbf{m}_t}{\sqrt{\mathbf{v}_t} + \epsilon}$ (with bias correction if needed). This is the standard adaptive moment update (with Nesterov already accounted for in $\mathbf{m}_t$). It scales each parameter's momentum by the inverse root of its recent gradient variance or curvature, so parameters with volatile or high-curvature gradients get smaller steps.
14. **Lion style (sign-based):** $\Delta \theta_t = -\eta \frac{\text{sign}(\mathbf{m}_t)}{\sqrt{\mathbf{v}_t} + \epsilon}$ (plus perhaps a small term $-\eta \lambda \theta_{t-1}$ for weight decay as Lion does ¹⁶). This ignores $\mathbf{v}_t$ entirely and uses only the sign of momentum, giving all parameters the same step size (scaled by learning rate) irrespective of gradient magnitude. It's very memory efficient and can be robust.
15. **Hybrid approach:** One novel idea is to combine these: use the **sign of $\mathbf{m}_t$ for direction** but modulate the step size by $1/\sqrt{\mathbf{v}_t}$ or a function of $\mathbf{v}_t$. I.e., $\Delta \theta_t = -\eta f(\mathbf{v}_t) \cdot \text{sign}(\mathbf{m}_t)$, where $f(\mathbf{v}_t)$ could be $(1/\sqrt{\mathbf{v}_t + \epsilon})$ clipped to a reasonable range. This way, in directions where $\mathbf{v}_t$ (variance or curvature) is large (meaning high curvature or inconsistent gradients), $f(\mathbf{v}_t)$ is small, reducing the step magnitude, but we still largely ignore the exact gradient value (using sign). In flat/consistent directions (small $\mathbf{v}_t$), $f(\mathbf{v}_t)$ is larger, allowing bigger steps. This hybrid is speculative but aims to get the best of both worlds: the **robust directionality** from sign updates and the **adaptive scaling** from second-order info. It also inherently caps how large any single update can get (since even if $f(\mathbf{v}_t)$ is large, the sign update magnitude might be bounded by some max factor or separate clip).
16. **Weight Decay (Regularization):** Apply decoupled weight decay as a separate step: $\theta_t = \theta_{t-1} + \Delta \theta_t - \eta \lambda \theta_{t-1}$. (In Lion's update, the weight decay was folded into the update term as $\lambda \theta$ inside the sign[] ¹⁶, which is equivalent to decoupled decay for small steps.) Weight decay helps generalization by penalizing large weights and is important in tasks like classification. We will tune λ such that it doesn't overshadow the learning on segmentation/detection tasks that might be more sensitive to weight scale.
17. **Lookahead Step:** Every K iterations (e.g., $K=5$), perform the lookahead update: Let θ^{slow} be a copy of model parameters that updates more slowly. After K fast updates to θ (as above), update $\theta^{\text{slow}} := \theta^{\text{slow}} + \alpha (\theta - \theta^{\text{slow}})$, with $\alpha \approx 0.5$. Then set $\theta := \theta^{\text{slow}}$ (synchronize) for the next iteration. This "pulls back" the fast weights towards a more stable position and can improve convergence stability ³⁶. If using lookahead, θ^{slow} is what we ultimately evaluate on and use for the next tasks' gradient computation. Lookahead adds little overhead (just one extra copy of parameters in memory

and an interpolation calculation) but can **yield smoother and often faster convergence** ¹³. We will compare runs with and without lookahead in experiments.

18. **Hyperparameter Adaptation (optional):** Monitor metrics and automatically adjust hyperparameters: for instance, if overall loss plateaued and started increasing, reduce learning rate (or use a predefined schedule). Or use hypergradient: $\eta_{t+1} = \eta_t - \alpha \frac{\partial L}{\partial \eta}$ as per hypergradient descent ²⁴ – though computing that derivative exactly is tricky, simple heuristics or proportional adjustment can simulate it. Another idea: adjust task loss weights based on performance (if one task consistently underperforms, increase its weight). These meta-adjustments ensure the optimizer doesn't require exhaustive manual tuning for each new multi-task setup.

Summarizing the **MTOpt algorithm** in a pseudo-code style:

```
Initialize  $\theta$ , momentum  $m=0$ , second-moment  $v$  (or Hessian estimate)  $= 0$ , slow weights  $\theta^{\text{slow}} = \theta$  (for lookahead)

For  $t = 1$  to  $\text{max\_iters}$ :
    1. Compute task gradients  $g_i = \nabla_{\theta} L_i(\theta)$  for each task  $i = 1 \dots T$ 
    2. For each pair  $(i,j)$ : if  $g_i \cdot g_j < 0$ , set  $g_i := g_i - (g_i \cdot g_j / ||g_j||^2) * g_j$  # PCGrad
    3. Optionally adjust magnitudes of  $g_i$  (GradNorm or loss weighting)
    4. Combine gradients:  $g = \sum_{i=1}^T g_i$ 
    5.  $m := \beta_1 * m + (1-\beta_1) * g$  # momentum update (Nesterov can be applied by using  $\theta + \beta_1 * m$  in grad computation)
    6. If using adaptive  $v$ :
         $v := \beta_2 * v + (1-\beta_2) * \varphi(g)$  #  $\varphi(g) = g^2$  (Adam), or  $(g - m)^2$  (AdaBelief), or Hessian_diag (every  $k$  steps)
    7. Determine update  $\Delta\theta$ :
        If using Adam-style:  $\Delta\theta := -\eta * m / (\sqrt{v} + \epsilon)$ 
        If using Lion-style:  $\Delta\theta := -\eta * \text{sign}(m)$  (or hybrid:  $-\eta * (1/\sqrt{v}) * \text{sign}(m)$ )
    8.  $\theta := \theta + \Delta\theta$  # gradient step
    9.  $\theta := \theta - \eta * \lambda * \theta$  # decoupled weight decay (if  $\lambda > 0$ )
    10. Every  $K$  steps (lookahead):
         $\theta^{\text{slow}} := \theta^{\text{slow}} + \alpha * (\theta - \theta^{\text{slow}})$ 
         $\theta := \theta^{\text{slow}}$  # sync weights
```

This optimizer integrates **adaptive moment estimation, Nesterov momentum, gradient conflict resolution, and second-order scaling**. Several components (like the exact form of v update or whether to use sign or magnitude) can be toggled based on empirical performance. In essence, MTOpt reduces to Nadam (momentum + adaptive) if we disable the advanced features, but it has the capacity to leverage **curvature (for speed), gradient surgery (for multi-task balance), and sign momentum (for memory efficiency and robustness)** to surpass Nadam's performance.

Memory Footprint: MTOpt stores: one momentum vector m (same size as parameters) and one v vector (same size) for adaptivity. This is identical to Adam’s memory cost – about 2× the model size. For a moderate CNN (say 10 million parameters for CIFAR-10 model), and using 32-bit floats, that’s on the order of 80 MB for $m+v$, well within a 12GB GPU budget. If memory is a concern, strategies like **Adafactor** (factorized second moment) or channel-wise v (as in Apollo optimizer) could be used. In fact, the recent **Apollo optimizer** showed that you can compress the second-moment state to extremely low dimensions (per-channel or even one value per tensor) and still match AdamW performance ^{7 37}. Apollo’s extreme version stored only a rank-1 scaled value per tensor and achieved *SGD-like memory usage with AdamW-level accuracy* ^{7 38}. This kind of approach, if needed, could allow training even larger models. For example, **Apollo enabled training a 7B-parameter model (LLaMA) from scratch on a single GPU with <12GB memory (with quantization)** ³⁹, highlighting how state storage can be drastically reduced. For our case, we likely don’t need such aggressive compression, but it’s good to know the design can scale in that direction if we attempt larger models or larger batch sizes.

Implementation Feasibility and Architecture

To implement the proposed optimizer on a single RTX 3060, we will use a framework like PyTorch which allows custom optimizer definitions. Key considerations:

- **Custom Optimizer Class:** We can extend `torch.optim.Optimizer` to create MTOpt. This involves writing the update rule in code following the steps above. PyTorch autograd will compute the gradients for each loss; we then apply our modifications and update the parameters. We will be careful to implement efficient operations (e.g., use in-place ops where possible to save memory, and vectorize the PCGrad projections).
- **Efficiency:** The heaviest extra computation in our optimizer is the optional Hessian diagonal estimation (Sophia step). Doing this every 10 steps (for example) means an occasional extra backward pass. On CIFAR-10 models, this overhead is minor (the model is small); 5-10% extra computation is acceptable ²⁰. If it proves too slow, we can reduce frequency or use a simpler curvature estimate (like gradient squared which we already compute).
- **Memory:** As analyzed, two state vectors of model size are fine. If we add lookahead, storing a copy of parameters (another vector of model size) is also fine (another ~40 MB). The gradient balancing step stores per-task gradients – if T tasks, that’s T gradient tensors. For $T=3$ tasks, that’s triple the gradient memory during the optimizer step, which is manageable. We will ensure to delete or release per-task grad tensors after use to avoid lingering memory use.
- **Existing Libraries:** We will draw from existing optimizers for reference. For instance, PyTorch’s AdamW source for how to organize momentum and weight decay, and perhaps Open-source implementations of PCGrad and lookahead. In fact, libraries like **pytorch-optimizer** already implement many of these components (it lists optimizers like Lion, Apollo, AdaHessian, Ranger21, etc. and even PCGrad as an augmentation) ^{40 41}. We can reuse or adapt those proven implementations to avoid bugs. This also means our implementation effort is feasible without needing to code everything from scratch in isolation – we are essentially *composing* known techniques with some novel glue logic.
- **Code Example:** To illustrate, a simplified PyTorch-style pseudo-code for the core update might look like:

```

for group in optimizer.param_groups:
    for p in group['params']:
        # Assuming we stored per-task gradients externally or in p.grad (for
        combined)
        grad = p.multitask_grad # our computed combined gradient
        if grad is None: continue
        # Update momentum
        state = optimizer.state[p]
        m = state['momentum']
        v = state['variance']
        m.mul_(beta1).add_(grad, alpha=1-beta1)
        if use_adaptive:
            # AdaBelief or Adam style second moment
            diff = grad - m if use_belief else grad
            v.mul_(beta2).addcmul_(diff, diff, value=1-beta2)
        # Compute update
        if use_sign_update:
            update = m.sign() # sign of momentum
            if use_adaptive:
                update = update * (v.add(eps).sqrt().reciprocal()) # scale by
1/sqrt(v)
        else:
            update = m / (v.add(eps).sqrt()) # Adam-style
        # weight decay
        if weight_decay:
            p.data.mul_(1 - lr*weight_decay)
        # parameter update
        p.data.add_(update, alpha=-lr)

```

This is a high-level sketch; actual implementation will handle bias-corrections (for momentum initially) and ensure operations are properly done on tensors. The use of `addcmul_` in updating `v` is a vectorized way to do $v = \beta_2 v + (1 - \beta_2) \text{diff}^2$. The `p.multitask_grad` would be prepared by our PCGrad routine beforehand.

- **Debugging and Tuning:** We will include logging to monitor the norms of gradients, momentum, and updates for each task and overall. This will help verify that, for example, PCGrad is indeed reducing conflicts (we can monitor angle between task gradients), and that adaptive scaling is working (e.g., check how many updates get clipped or how second moment evolves).
- **Verification on Simpler Problems:** Before tackling the full multi-task CNN, we will test the optimizer on simple known problems (like a single-task CIFAR-10 training to ensure it behaves at least as well as Adam/Nadam). We could also simulate a multi-task on CIFAR by splitting classes (two classification tasks on different splits of CIFAR-10) to see if gradient balancing works. These tests ensure correctness.

Overall, implementation is feasible and can largely build on known components. Many features of MTOpt have been implemented and tested in isolation by prior work (e.g., code for AdaBelief ¹¹, Lion, lookahead, PCGrad are available), which de-risks our development.

Evaluation Strategy

To assess the optimizer’s performance relative to Nadam, we will carry out a series of experiments:

- **Dataset and Tasks:** Use CIFAR-10 as the base dataset for classification. To introduce segmentation and detection, we have a few options since CIFAR-10 doesn’t natively have those annotations. One approach is to create auxiliary tasks on CIFAR-10: for example, treat the image classification label as one task, and a synthetic segmentation task such as segmenting foreground vs background (we could generate rough segmentation masks via an unsupervised method or use the object labels to create a coarse mask). For detection, perhaps create bounding boxes around the object in the image (CIFAR-10 images often have a centered object). If synthetic tasks are not ideal, we could evaluate on a multi-task dataset like Cityscapes (which has classification attributes, segmentation, and could treat detection of certain classes as another task) to truly test classification/segmentation/detection together. However, that might require a different backbone than a simple CIFAR CNN. For initial experiments, a **multi-head CNN** on CIFAR-10 can be trained to predict: class label, a segmentation mask (even if approximate), and bounding box of the object. The exact multi-task setup will be clearly defined in the experimental protocol.
- **Baselines:** Compare our **MTOpt optimizer** to Nadam, as well as other optimizers for context: AdamW (a strong baseline), SGD with momentum (to see generalization differences), and perhaps Lion and Sophia if available, to gauge if our combination truly excels. Each optimizer will train the same model architecture on the same tasks for a fair comparison, with hyperparameters tuned as well as possible (learning rates, etc.).
- **Metrics:** For **training speed**, measure the number of epochs (or wall-clock time) required to reach certain performance thresholds on each task (e.g. 85% classification accuracy, or certain segmentation IoU). We expect MTOpt to require fewer epochs than Nadam for the same accuracy due to better-curated updates ⁴. Also track *time per iteration*, though if our optimizer has slightly more overhead, the key is whether it converges in significantly fewer iterations to offset that. For **accuracy**, use standard metrics: classification accuracy (%), segmentation mean Intersection-over-Union (mIoU) or pixel accuracy, and detection mean Average Precision (mAP) if applicable. The multi-task optimizer should ideally improve or match all these metrics relative to Nadam. If one task is slightly worse but others much better, we’ll analyze the trade-off. For **generalization**, evaluate the tasks on a separate test set (e.g., CIFAR-10 test for classification, and appropriate test sets for the other tasks). Possibly also test robustness by introducing domain shift (like CIFAR-10 images with noise or different brightness) to see if the optimizer’s solution is more robust. Generalization can be indirectly assessed by metrics like the gap between training and testing performance – smaller gaps usually indicate better generalization (provided similar training final losses). For **memory efficiency**, record GPU memory usage during training for each optimizer. We expect MTOpt to use less memory than Nadam if we have incorporated memory-saving measures (e.g., if we turned on a feature to compress state). If using full Adam-like state, memory might be similar; however, if our optimizer converges faster, we might handle larger batch sizes or higher resolution within the same memory as a side benefit. We will also measure if any CPU offloading or computation is needed for second-order steps.

- **Ablation Studies:** To understand which components contribute most, we will run ablations of MTOpt:

- MTOpt without PCGrad/GradNorm (i.e., treat it like a vanilla single-task optimizer on the summed loss) to see how much the multi-task balancing helped. We expect to see possibly slower or unstable training in that case, or one task dominating.
- MTOpt without second-order (no Hessian, just Adam-style v) vs with Sophia-style curvature. This will show if the curvature estimation speeds up convergence as hypothesized.
- MTOpt with sign-update vs magnitude update. This will clarify the effect on generalization and memory. Sign-based updates might converge slightly slower but generalize better, according to Lion's results ¹⁸, so it's interesting to observe.
- Lookahead vs no-lookahead. This often helps final accuracy, so we expect a modest gain in stability. Each ablation is evaluated on the same metrics to pinpoint the contributions.

- **Statistical Significance:** We will run multiple trials (3 or more runs) for each setting because training can be stochastic. We'll report mean and variance of results (e.g., final accuracy) to ensure differences are real and not just lucky initializations. If MTOpt has more hyperparameters (β_1 , β_2 , etc.), we'll ensure we tune them reasonably (perhaps using a small grid or referring to values from literature like $\beta_1=0.9$, $\beta_2=0.999$ for Adam, which generally work well).

- **Evaluation of Generalizability:** One specific test for generalization is to examine the *sharpness* of the final solution. We can plot or measure the loss increase if weights are perturbed (flat minima vs sharp minima). SAM-like techniques or the presence of second-order info might yield flatter minima. Another test is to evaluate on slightly different tasks: e.g., train on CIFAR-10 multi-task, test on CIFAR-100 classification or CIFAR-10 with some corruption, to see if any improvement carries over. While this is not a standard requirement, it may highlight if our optimizer's solution is more robust (a sign of better generalization).

- **Memory/Compute Evaluation:** To ensure the optimizer is practical on RTX 3060, we will measure per-iteration time and memory. If we find the second-order computations too slow, we might try to optimize them (e.g., compute curvature less frequently). If memory is tight at any point, we might reduce state (though as argued, it should be fine). The single GPU nature also means we might try larger batch sizes to see if the optimizer remains stable (since some optimizers like Adam can behave differently with batch size; our method with adaptive and gradient clipping hopefully handles larger batches too).

Expected Outcomes: We anticipate MTOpt to show faster initial loss decrease than Nadam (due to better-curated gradient directions and potentially second-order jump-start), and to reach a higher or equal plateau of performance on all tasks. For example, it might reach say 92% classification accuracy where Nadam stops at 90%, or achieve that accuracy in half the epochs. For segmentation/detection (if our setup is reasonable), we expect improved metrics or at least much faster attainment of peak performance. In terms of memory and time, MTOpt might use a bit more time per iteration (if using Hessian calc) but fewer iterations total, resulting in net training time savings. Memory usage should be on par or better than Nadam especially if sign-update or compressed states are used (Nadam and Adam both use 2 state vectors per param, which we also use unless we optimize it). A key thing to monitor is stability: multi-task training can be finicky (one task might explode in loss if not handled). With PCGrad and our adjustments, we expect a **smoother**

training curve – fewer spikes in loss when tasks conflict – compared to Nadam which might oscillate when one task’s gradient opposes another’s. If we see smoother curves and better final results, that would confirm the effectiveness of our design.

Discussion of Research Directions

Our design draws on state-of-the-art ideas in optimizer research. To contextualize, some recent optimizers and techniques that influenced us include:

- **Ranger21**: An optimizer that **combines AdamW with multiple enhancements** (gradient centralization, lookahead, adaptive momentum, etc.) ⁴². It achieved improved accuracy and training speed on ImageNet, even training ResNet50 without BatchNorm by avoiding bad initial states ¹³. This shows that a *synergistic combination* of optimizer tricks can outperform standard optimizers. Our approach is analogous in spirit – integrating several proven techniques (with references to their theoretical foundations) to handle our specific use case.
- **Lion and Sophia**: These recent optimizers represent two extremes – Lion is first-order with minimal memory (using sign momentum) and Sophia is second-order with a bit more computation. Interestingly, both yield faster training and good results ¹⁸ ⁴. There is ongoing research into hybrids or improvements of these (e.g., variants of Lion that incorporate some curvature information, or making second-order methods more robust). Our proposed hybrid sign+adaptive update is a novel angle to get benefits of both. It would be valuable to follow-up on how this idea works in practice.
- **Meta-Optimizers**: Work by Google Brain and others on learned optimizers suggests that as tasks get more complex (like multiple objectives), there might be non-intuitive update rules that work better than classical methods. For instance, a learned optimizer could in theory discover task-specific adaptive schedules or interactions. We referenced such work but our current design is hand-crafted. A future research direction is to **meta-train the hyperparameters or components** of our optimizer on a family of multi-task problems. For example, one could use a smaller proxy multi-task dataset to learn the best way to blend sign and magnitude updates, or to learn a schedule for β_1 , β_2 over training time. This could further boost performance and adaptivity.

Additionally, our optimizer currently treats the *combined loss* in a fairly standard way (after PCGrad). Another frontier is **multi-objective optimization** theory, which might seek a set of Pareto-optimal solutions rather than a single combined optimum. Methods like evolutionary algorithms or game-theoretic formulations (treating each task as a player) exist. Those are beyond our scope, but if truly optimal balance between tasks is required, one could incorporate concepts like *Nash bargaining solution* for tasks or alternate updates focusing on one task at a time in a round-robin (some have tried that to ensure each task gets dedicated attention). We chose PCGrad and GradNorm as practical mechanisms with low overhead and proven gains ² ³².

Conclusion and Next Steps

We have outlined the design of a novel optimizer, **MTOpt**, tailored for multi-task learning on CNNs (with CIFAR-10 as a running example). By combining **adaptive learning rates** (for fast convergence), **momentum/Nesterov acceleration** (to smooth updates), **second-order curvature estimates** (to intelligently scale steps for speed and stability), and **gradient surgery and normalization** (to handle multi-task conflicts and imbalance), the proposed optimizer addresses many limitations of Nadam in the multi-

task context. The design emphasizes **training speed improvements** (through curvature-informed jumps and reduced interference), aims for **high accuracy on all tasks** (through balanced updates), seeks **better generalization** (via weight decay, possibly flatter minima from second-order info, and careful step clipping), and maintains **memory efficiency** (no worse than Nadam, with options to reduce state).

We backed these choices with references to state-of-the-art research. For example, PCGrad's ability to improve multi-task optimization ³⁰ gives us confidence in the gradient surgery component, while Sophia's success in halving training steps for similar accuracy ⁴ suggests our curvature approach can pay off. Lion's memory-saving sign updates ⁶ inspire our consideration for using sign-based techniques to keep the optimizer lightweight.

Next Experimental Steps:

1. **Implement and Unit Test MTOpt:** Begin coding the optimizer in a controlled environment. Test on a simple known function (e.g., a 2D quadratic or a small neural net on one task) to verify basic behavior (convergence, correct application of updates like weight decay and lookahead).
2. **Tune Hyperparameters on CIFAR-10 Single Task:** Before multi-task, ensure MTOpt is at least as good as Adam/Nadam on CIFAR-10 classification alone. Adjust learning rate, β_1 , β_2 , etc., and confirm the curvature estimation frequency/clipping doesn't cause issues. This establishes a baseline of proper functioning.
3. **Multi-Task Training on CIFAR-10:** Train the multi-task CNN with MTOpt vs Nadam (and others). Monitor training curves, and iterate on hyperparameters: for example, the relative task loss weights if needed, the PCGrad application frequency (we can even apply PCGrad not on every step if tasks are numerous; in our case it's fine to do every step). Check for any instability – if one notices oscillations, consider increasing β_2 (more smoothing) or decreasing learning rate, or increasing clipping. Our optimizer has more moving parts, so some tuning is expected. We should use the ablation studies to decide if all components are needed or if a simpler variant works nearly as well.
4. **Analyze Results:** Evaluate final metrics as described. If the optimizer meets or exceeds our expectations (faster convergence, higher combined task performance than baselines), that's a success. If not, analyze which component might be underperforming. For example, if second-order estimation isn't helping, maybe the problem size is too small to need it – we could drop it. Or if one task still lags, perhaps more aggressive GradNorm or a different task weighting is required. The design is flexible, so we can iterate.
5. **Memory/Compute Profile:** Ensure that the training fits in 12GB with comfortable headroom. Profile the runtime to see where the most time is spent (maybe the Hessian estimation – could we skip more steps? Or the PCGrad projection – though that's quick for 3 tasks). If needed, optimize these (e.g., use lower precision for certain computations, which is viable now that GPUs support mixed precision – momentum and v could potentially be FP16 to save memory, as long as stability holds).
6. **Extend to Other Scenarios:** Once validated on CIFAR-10, a logical next step is to try a more realistic multi-task dataset (like NYU v2 for multi-task depth/segmentation/classification or Cityscapes for seg & detection). Our optimizer design should generalize to other CNN-based multi-task problems, but testing it will prove its robustness. If those tasks are larger, it will also test the memory efficiency (we may then employ the Apollo-style memory saver).
7. **Open-Source and Community Feedback:** Since optimizers are of broad interest, we can release the MTOpt implementation along with our experimental results. The community (researchers, developers) could try it on their tasks, providing feedback or improvements. Given that our approach is built on modular ideas, others might swap in newer techniques (say, a different meta-learning schedule) or apply it to single-task but large-scale problems to see if the combination has merits there too.

In conclusion, designing an optimizer from scratch allowed us to combine cutting-edge techniques to specifically address multi-task CNN training. The proposed optimizer is ambitious in integrating many components, but each piece is grounded in theory or empirical success from recent studies. If successful, this optimizer would provide a valuable tool for training multi-task networks more efficiently on accessible hardware. Even if some aspects don't yield expected gains, the exercise illuminates which innovations truly make a difference in complex optimization scenarios. With a thorough evaluation and iteration process, we aim to demonstrate **improved training speed, accuracy, and robustness** over Nadam, contributing to the state of the art in optimizer design for multi-task deep learning.

References:

- Yu, Tianhe, et al. "Gradient Surgery for Multi-Task Learning." NeurIPS 2020. ² ³⁰
- Chen, Zhao, et al. "GradNorm: Gradient Normalization for Adaptive Loss Balancing in Deep Multitask Networks." ICML 2018. ³ ²⁶
- Zhuang, Juntang, et al. "AdaBelief Optimizer: Adapting Stepsizes by the Belief in Observed Gradients." NeurIPS 2020. ⁵ ¹¹
- Liu, Liu et al. "On the Variance of the Adaptive Learning Rate and Beyond – Adam, RAdam, etc." ICLR 2020. ⁴³
- Chen, T., et al. "Symbolic Discovery of Optimization Algorithms." 2023 (Lion Optimizer). ²³ ¹⁶
- Zhang, et al. "Lookahead Optimizer: k steps forward, 1 step back." NeurIPS 2019. ¹²
- Luo, etc. "Adaptive Gradient Methods with Dynamic Bound of Learning Rate." ICLR 2019 (AdaBound).
- Liu, et al. "Variance Reduction in Adam (RAdam)." 2019. ⁴³
- Berrada et al. "Sharpness-Aware Minimization for Efficiently Improving Generalization." ICLR 2021.
- Gidel et al. "A Variational Inequality Perspective on Generative Adversarial Nets." (For extragradient methods relevant to optimism in updates.)
- Botev et al. "Batch Normalization Eliminates Singularity of the Hessian in Deep Networks." (Related to curvature and conditioning).
- Deng et al. "Sophia: A Scalable Second-Order Optimizer for Language Model Pre-training." ICLR 2024. ¹⁹ ²¹
- Zhao et al. "Apollo: Approximated Gradient Scaling for Memory-Efficient LLM Optimization." MLSys 2025. ⁷ ³⁹
- Baydin et al. "Online Learning Rate Adaptation with Hypergradient Descent." 2017. ⁴⁴
- Andrychowicz et al. "Learning to learn by gradient descent by gradient descent." 2016.

(Additional citations embedded in text above.)

¹ ² ²⁷ ³⁰ ³¹ proceedings.neurips.cc

<https://proceedings.neurips.cc/paper/2020/file/3fe78a8acf5fda99de95303940a2420c-Paper.pdf>

³ ²⁶ ³² ³³ [1711.02257] GradNorm: Gradient Normalization for Adaptive Loss Balancing in Deep Multitask Networks

<https://arxiv.org/abs/1711.02257>

⁴ ¹⁹ ²⁰ ²¹ ²² ³⁴ ³⁵ Paper Summary #9 - Sophia: A Scalable Stochastic Second-order Optimizer for Language Model Pre-training | Shreyansh Singh

https://shreyansh26.github.io/post/2023-05-28_sophia_scalable_second_order_optimizer_llms/

5 8 10 11 [2010.07468] AdaBelief Optimizer: Adapting Stepsizes by the Belief in Observed Gradients
<https://arxiv.org/abs/2010.07468>

6 16 17 18 23 Lion Optimizer: Deep Learning & Swarm Methods
<https://www.emergentmind.com/topics/lion-optimizer>

7 9 37 38 39 [width=0.03]Figures/rocket.png APOLLO: SGD-like Memory, AdamW-level Performance
<https://openreview.net/pdf/cd881d9acd095aaf2a89081b3731610af07dd16a.pdf>

12 Reviews: Lookahead Optimizer: k steps forward, 1 step back
<https://proceedings.neurips.cc/paper/2019/file/90fd4f88f588ae64038134f1eaa023f-Reviews.html>

13 14 42 [2106.13731] Ranger21: a synergistic deep learning optimizer
<https://arxiv.org/abs/2106.13731>

15 ADAM in 2019 — What's the next ADAM optimizer | by Nok Chan
<https://medium.com/data-science/adam-in-2019-whats-the-next-adam-optimizer-e9b4a924b34f>

24 44 [PDF] online learning rate adaptation with hypergradient descent
<https://gbaydin.github.io/assets/pdf/baydin-2018-hypergradient.pdf>

25 gbaydin/hypergradient-descent - GitHub
<https://github.com/gbaydin/hypergradient-descent>

28 Conflict-Averse Gradient Descent for Multi-task Learning - arXiv
<https://arxiv.org/html/2110.14048v2>

29 [PDF] Conflict-Averse Gradient Descent for Multi-task Learning
https://proceedings.neurips.cc/paper_files/paper/2021/file/9d27fdf2477ffbf837d73ef7ae23db9-Paper.pdf

36 Implementation of the latest "Lookahead Optimizer" paper in Keras?
<https://stackoverflow.com/questions/57195278/implementation-of-the-latest-lookahead-optimizer-paper-in-keras>

40 41 pytorch_optimizer · PyPI
https://pypi.org/project/pytorch_optimizer/

43 On the Variance of the Adaptive Learning Rate and Beyond - arXiv
<https://arxiv.org/abs/1908.03265>